Trabajo práctico N.º 8 - Food Store

Base de Datos II · Unidad 8 · Informe de pruebas

Fecha: 8 de noviembre de 2026

Introducción

En este trabajo armamos una base SQLite para representar una tablet que se usa en una feria sin conexión a Internet. La idea es que pueda consultar los productos y guardar pedidos como borradores. Cuando vuelve a tener conexión, esos pedidos se comparan con la información del servidor para detectar cambios en precios, falta de stock o productos dados de baja.

Trabajamos con una copia de PostgreSQL llamada **foodstore_tp8**. La base original quedó separada de las pruebas. La tablet no confirma ventas ni modifica el stock de PostgreSQL: eso sigue siendo responsabilidad del servidor.

1. Base de datos de la tablet

1.a Datos que se guardan localmente

Se copiaron el identificador y nombre de cada producto, el precio en centavos, el stock visto por la tablet, las marcas de disponibilidad y baja lógica, y la fecha de la copia. No se incluyeron contraseñas, correos, información personal de clientes ni permisos de usuarios. Esos datos no hacen falta para consultar el catálogo.

1.b Tablas y restricciones

En SQLite se crearon las tablas **producto_cache**, **pedido_borrador**, **item_borrador** y **cola_sync**, todas con **STRICT**. Se agregaron claves primarias, claves foráneas y restricciones CHECK. Los precios se guardan como números enteros en centavos para evitar problemas al comparar importes. También se creó una vista para consultar solamente el catálogo disponible.

1.c Pruebas de transacciones e integridad

Para probar las transacciones utilizamos Python con el módulo sqlite3, porque DB Browser maneja sus propias transacciones. Las pruebas se hicieron sobre una copia de la base y no cambiaron el archivo original.

- **BEGIN y COMMIT:** después de guardar un pedido, un ítem y una solicitud, los conteos fueron (1, 1, 1).
- **BEGIN y ROLLBACK:** al deshacer la misma operación, los conteos fueron (0, 0, 0).
- **C1:** la consulta del catálogo disponible funcionó directamente en SQLite, sin conectarse al servidor.
- **C7:** una cantidad igual a cero fue rechazada por CHECK (código 275) y un producto inexistente por FOREIGN KEY (código 787). No se agregaron ítems inválidos.

2. Sincronización con PostgreSQL

2.a Copia del catálogo

En PostgreSQL creamos la vista **v_snapshot_movil** y exportamos los productos a un CSV. Después importamos ese archivo en SQLite y cargamos el catálogo local. La instantánea inicial se obtuvo el 8 de noviembre de 2026 y contiene **50.010 productos**. Los cambios posteriores de los casos C2, C3 y C4 se guardaron en nuevas instantáneas para poder compararlos.

2.b Funcionamiento del semáforo

El script de sincronización revisa las solicitudes que todavía están en estado PENDIENTE. Si un producto no existe, está dado de baja, no está disponible o no alcanza el stock, la solicitud queda RECHAZADA. Si esos controles pasan pero cambió un precio, queda en CONFLICTO. Cuando todo coincide, queda SIMULADO_APROBADO. El script guarda un motivo para cada resultado y no vuelve a evaluar las solicitudes ya procesadas.

2.c Casos de prueba

C2 - Cambio de precio. El borrador tenía Coca Cola 2L a $2.500 y en PostgreSQL se cambió a $2.800. El resultado fue CONFLICTO. Luego se actualizó el precio visto en el borrador y se creó una nueva solicitud con otro UUID. Ese intento terminó en SIMULADO_APROBADO.

C3 - Stock insuficiente. Se preparó un pedido por 10 unidades de Sprite 2L, pero el stock del servidor era 4. El resultado fue RECHAZADO.

C4 - Baja lógica. Se marcó como inactiva Agua Mineral 1.5L. El pedido fue RECHAZADO y el producto dejó de mostrarse en el catálogo disponible. La fila se mantuvo en la caché con la marca de eliminado.

C5 - Sin cambios. El pedido incluía dos Coca Cola 2L y una Sprite 2L. Como los datos coincidían con la instantánea utilizada en esa prueba, terminó en SIMULADO_APROBADO.

C6 - UUID repetido. Intentamos agregar otra solicitud con un UUID ya existente. SQLite rechazó la inserción por clave primaria (código 1555). La solicitud original se conservó y una nueva ejecución del semáforo no alteró los estados cerrados.

Al terminar, **cola_sync** conservó cinco solicitudes: dos SIMULADO_APROBADO, una CONFLICTO y dos RECHAZADO. Esto coincide con lo que se comprobó al abrir la base SQLite entregada.

Control de la base del servidor

También consultamos la cantidad de pedidos en PostgreSQL. Tanto **foodstorecopia** como **foodstore_tp8** mostraron **200.004 pedidos**. En la base de pruebas se observaron Coca Cola con stock 20, Sprite con stock 4 y Agua Mineral con stock 25 e inactiva. El stock de Sprite y la baja de Agua Mineral se modificaron manualmente para preparar los casos; no fueron cambios provocados por la simulación.

Por último, se revisó la base SQLite original: **PRAGMA integrity_check** devolvió **ok** y **foreign_key_check** no informó errores. La base tiene 50.010 productos, cuatro borradores, cinco ítems y cinco solicitudes de sincronización.

3. Integración con las unidades anteriores

3.a Qué guarda cada motor

Motor	Datos que conserva	Responsabilidad
PostgreSQL	Productos, precios, stock oficial y pedidos.	Es la fuente de verdad y el único que puede confirmar una venta.
SQLite	Catálogo copiado, borradores, ítems y cola de sincronización.	Permite trabajar sin conexión, pero no actualiza el servidor.
MongoDB	Documentos de pedidos con sus ítems, trabajados en la Unidad 7.	Se usa para consultas documentales; no controla stock ni ventas.

El recorrido de los datos es sencillo: PostgreSQL genera el CSV, SQLite lo utiliza para consultar el catálogo y guardar pedidos pendientes, y después se comparan esos pedidos con una instantánea más reciente. MongoDB queda como una representación documental de pedidos trabajada en la unidad anterior.

3.b Mantenimiento y recuperación

Como continuación del plan de la Unidad 6, proponemos realizar respaldos diarios de PostgreSQL y comprobarlos periódicamente con una restauración en una base aparte. También corresponde respaldar la información documental de MongoDB y guardar una copia consistente del archivo SQLite, sobre todo cuando tiene borradores pendientes.

Si PostgreSQL se restaura a un estado anterior, las tablets deberían detener los reintentos de sincronización hasta actualizar el catálogo y revisar los UUID y pedidos pendientes. De esa manera se evita enviar dos veces una operación que podría haberse confirmado antes de la restauración.

Las solicitudes en CONFLICTO o RECHAZADO deben revisarse al recuperar la conexión y al cierre de la jornada. El operador identifica los casos y el responsable del negocio o de la base decide si corresponde actualizar el precio, corregir cantidades o descartar el borrador.

3.c Revisión de scripts de la cursada

Se hizo una revisión preliminar de los archivos disponibles en el repositorio del proyecto. Encontramos estos puntos para analizar con OpenCode:

- En **TP4_unidad2/views.sql** aparece la línea **-VERIFIACIÓN**, con un solo guion. No es un comentario SQL válido y debe corregirse a **-- VERIFICACIÓN**.
- En la Unidad 5, el script inicial de roles concedía UPDATE general sobre **detalle_pedido**. Un script posterior restringió ese permiso por columnas. La documentación de esa unidad indica que **app_web** ya no puede actualizar **precio_unitario**.
- En **TP3_unidad2/indices.sql** se usa CREATE INDEX sin IF NOT EXISTS. No lo consideramos un error en una instalación que se ejecuta una sola vez, aunque sí puede dificultar una segunda ejecución. Agregar IF NOT EXISTS sería una mejora de mantenimiento, no una corrección indispensable.

Esta revisión todavía es **preliminar**. La consigna requiere utilizar **OpenCode en modo Plan** para revisar todos los scripts de la cursada, y comprobar los hallazgos en la base. Esa actividad no se presenta aquí como realizada.

4. Kiro, uso de IA y entrega

En la carpeta docs/kiro se prepararon los archivos **requirements.md**, **design.md** y **tasks.md** como propuesta de especificación. Los requisitos incluyen consulta sin conexión, guardado atómico, evaluación del semáforo y control de UUID repetidos. Falta registrar la ejecución real en Kiro, como pide la consigna.

Se utilizó ChatGPT como ayuda para interpretar la actividad, preparar SQL, resolver problemas de DB Browser y redactar documentación. Los resultados de las pruebas se contrastaron con la base y con las salidas observadas. La bitácora y el borrador de DUIA están en la entrega; deberán completarse con las interacciones reales de Kiro y OpenCode y con el formato oficial de la cátedra.

Conclusión

Las pruebas mostraron que la tablet puede conservar un catálogo y preparar pedidos sin conectarse al servidor. La comparación posterior permite detectar precios cambiados, stock insuficiente, bajas lógicas y solicitudes repetidas. El punto principal del trabajo es que un resultado SIMULADO_APROBADO no significa que se haya vendido el producto: el stock y las ventas siguen bajo control de PostgreSQL.

Material consultado: repositorio *Tomimartin/BaseDeDatos2* (unidades anteriores) y archivos de pruebas del TP8.